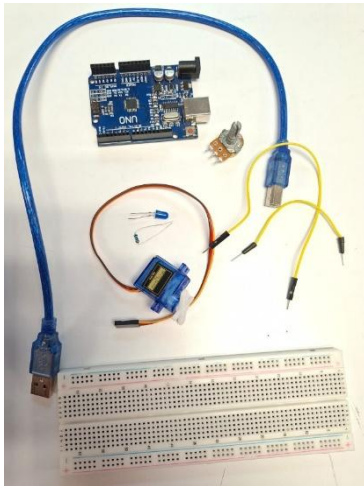


LABORATORIO 07 PMW y Control Analógico: Dimmer de Led y Control de Servo

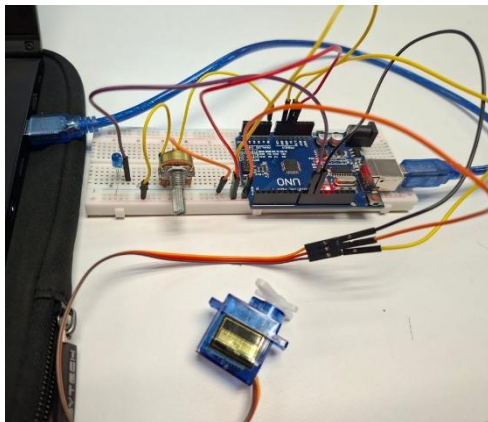
Objetivo: Generar señales PMW con un microcontrolador para controlar el brillo de un LED y la posición de un servomotor, comprendiendo la relación entre el ciclo de trabajo y el efecto físico resultante.

Materiales:

- Arduino UNO, Nano o ESP32 (cualquier plataforma con PWM)
- LED de cualquier color + resistencia 220 ohm
- Servomotor SG90 o MG90S (servo de 9g, muy económico)
- Potenciómetro de 10 kohm (para control manual del servo)
- Protoboard y jumpers
- Multímetro (opcional, para medir el voltaje promedio del PWM)



Circuito Armado



PREGUNTAS DE REFLEXION Y REPORTE

¿Cuántos niveles de brillo distintos puede generar analogWrite() en Arduino UNO?

¿Cuántos bits tiene el DAC implícito del PWM?

En Arduino UNO la función analogWrite() trabaja con valores de 0 a 255, por lo que puede generar 256 niveles distintos de brillo en un LED.

El PWM maneja una resolución de 8 bits.

Si el servo recibe un pulso de 1.5 ms cada 20 ms (50 Hz), ¿cuál es el duty cycle en porcentaje? ¿Y para un pulso de 1 ms?

El duty cycle se calcula dividiendo el tiempo encendido entre el periodo total.

Para 1.5 ms cada 20 ms:

$$20/1.5 \times 100 = 7.5\%$$

Para 1 ms cada 20 ms:

$$20/1 \times 100 = 5\%$$

Entonces:

$$1.5 \text{ ms} = 7.5 \%$$

$$1 \text{ ms} = 5 \%$$

¿Por qué el LED parece aumentar de brillo de forma no lineal cuando el duty cycle va de 0 a 255 de manera lineal? ¿Qué cambio en el código produciría una rampa de brillo más uniforme perceptualmente?

Esto ocurre porque la vista humana no percibe la luz de forma lineal. Aunque el PWM aumente en pasos iguales, nosotros notamos más cambio en niveles bajos que en niveles altos. Por eso, al subir de 0 a 255, el brillo parece acelerarse o cambiar de forma desigual.

Para corregirlo se puede aplicar una corrección gamma, que ajusta los valores antes de enviarlos al PWM.

Por ejemplo, en lugar de usar el valor directo, usar una fórmula como:

$$\text{valor} = \text{pow}(x / 255.0, 2.2) * 255;$$

Con esto el cambio de brillo se percibe más suave y uniforme.

El ADC del ATmega328P es de 10 bits. ¿Cuántos niveles distintos puede diferenciar? Si el rango es de 0 a 5V, ¿cuál es la resolución en mV por nivel?

Un ADC de 10 bits puede representar:

$$2^{10}=1024$$

Por lo tanto, puede diferenciar 1024 niveles distintos de 0 a 1023.

Si el rango es de 0 a 5V, la resolución por nivel es:

$$1024/5V=0.00488V$$

Eso equivale a:

4.88 mV por nivel

Esto significa que cada incremento en la lectura representa un cambio pequeño de voltaje.

¿Cómo modificarías el sketch para que al recibir el comando 'SCAN', el servo haga un barrido de 0 a 180 y de regreso, mientras reporta la posición cada 10 grados vía serial?

Se podría agregar un nuevo comando llamado SCAN dentro de procesarComando().

Cuando se reciba, el servo comenzaría a moverse desde 0 hasta 180 grados y luego regresaría a 0. Además, cada vez que llegue a múltiplos de 10 grados enviaría su posición por Serial.

La idea sería usar un ciclo como:

```
for (int i = 0; i <= 180; i += 10)
```

Y luego otro para regresar.

Mientras se mueve, enviaría mensajes como:

Posicion: 0

Hasta la posición: 180

Esto permitiría monitorear el movimiento del servo desde la computadora y verificar que el barrido se realiza correctamente.